

GridScript

Design and implementation report

A small interpreter for scripting a 2D game actor

Contributors

Nabil Ismail Abdulkadir

UG22CSC1047

Interpreter and semantics

Ahmad Auwal Abubakar

UG22CSC1075

Parser and grammar

Abubakar Muhammad Sulaiman

UG22CSC1046

Lexer and token rules

Rukayya Musbahu Imam

UG22CSC1040

AST and lexical scoping

Muhammad Salisu

UG20CSC1005

Tests and report

Overview

GridScript is a small imperative language for scripting a game actor on a 2D grid. It supports variables and arithmetic, `if` and `while` control flow, user-defined functions with parameters and return, and actor actions such as `step_forward()`. We kept the language small enough to specify, implement, and test during the project period.

The program is interpreted rather than compiled. `gridscript.py` reads a `.gsc` source file, tokenises it, parses it into an abstract syntax tree (AST), and evaluates that tree. The implementation applies the course topics of regular expressions and finite automata, context-free grammars, operational semantics, names and binding, and type systems.

Lexer, parser and evaluator

The lexer (`src/lexer.py`) is a handwritten scanner based on regular-expression patterns. It recognises numbers as `[0-9]+`, identifiers as `[a-zA-Z_][a-zA-Z0-9_]*`, strings, operators, and punctuation. A keyword table reserves the twelve language keywords. It skips comments beginning with `//`; an unexpected character such as `$` produces a lexer error before parsing starts.

The parser (`src/parser.py`) is a recursive-descent parser based on the BNF grammar in the design document. Its grammar levels enforce precedence: multiplication and division bind more tightly than addition and subtraction, so `2 + 3 * 4` becomes `2 + (3 * 4)`. It creates AST dataclasses rather than evaluating source text directly. [Annotated parser AST evidence](#) shows the parsed structure of a function definition.

The interpreter (`src/interpreter.py`) walks the AST using the evaluation rules in the design document. `set x = e` evaluates `e` and stores the result in the current environment. A function call evaluates its arguments, creates a fresh call environment, binds its parameters, runs the body, and returns the value supplied by `return`.

Scoping and typing

GridScript uses static (lexical) scoping. A function call creates a new environment whose parent is the environment captured when the function was defined. Reads search from the current environment outward, while `set` writes only in the current environment. A local `x` therefore shadows rather than overwrites a global `x`. The [annotated scoping evidence](#) shows the expected output `100, 7, 100`.

The language is dynamically typed. Values are Numbers, Strings, or Booleans, and the interpreter checks operators at run time. Arithmetic and `</>` require Numbers; `+` also concatenates two Strings. Conditions must be Boolean. Equality compares both value and GridScript type, so `1 == true` is `false` and `1 != true` is `true`.

Testing and group responsibilities

The integration runner checks eight valid programs against expected output and nine invalid programs against expected errors: all 17 pass. The direct test runner contains 65 focused checks for tokenisation, parsing, evaluation, recursion, lexical scoping, type errors, Boolean-versus-Number edge cases, and functions without return. [Annotated integration-test evidence](#) is available in Google Drive.

Nabil implemented the evaluator and built-ins; Ahmad implemented the parser and BNF grammar; Abubakar implemented the lexer; Rukayya implemented the AST and environment model; and Salisu implemented the test suites and assembled the report. `DIVISION_OF_LABOR.md` records each role and its course-topic link. [All source code, tests, and evidence are in the public repo](#).

AI-use disclosure

NotebookLM was used to brainstorm language themes and prepare slides for an initial group meeting. AI assistance was also used to clarify one Python error message during development. The group wrote and reviewed the grammar, operational rules, scoping and typing choices, and core implementation themselves.